

Nut AI Complete Quality Report

Every feature re-audited after the cycle that fixed the AI feature chain on aicredits.in, implemented the UI/UX transformation waves, and grew the suite to 1,266 green tests. Fifteen findings catalogued with live evidence, impact and fix; seven commits live-verified and pushed.

15

FINDINGS

4

P1 CRITICAL

6

P2 MAJOR

5

P3 MINOR

1266

TESTS GREEN

Nut AI Engineering

2 October 2026 · Baseline ca5b316 · 7 commits this cycle

Table of Contents

- 1. Executive Summary** 2
 - The five issues that most deserve the next working session, in order 3
- 2. Scope, Method and Severity Model** 3
- 3. AI and Reseller Reliability Audit** 4
 - 3.1 The measured gateway contract 4
 - 3.2 The fix: gateway wire mode 4
 - 3.3 Live verification matrix 5
 - 3.4 The assistant tool-misroute fix 6
- 4. P1 - Critical (4 findings)** 6
- 5. P2 - Major (6 findings)** 7
- 6. P3 - Minor and Polish (5 findings)** 9
- 7. Fixed This Cycle - Verification Ledger** 10
- 8. Performance and Size Audit** 11
 - 8.1 Bundle and corpora 11
 - 8.2 Boot and runtime 11
 - 8.3 Token economy 11
- 9. Test and Gate Audit** 12
- 10. Fix Roadmap** 12

1. Executive Summary

This report is the second complete quality audit of the Nut AI application - the Expo/React Native offline nutrition and training tracker with bring-your-own-key AI features - as of commit **ca5b316** on 2 October 2026. Where the first audit (30 September, baseline 1a711a9, 101 findings) catalogued a healthy engine behind a prototype-feeling interface, this cycle set out to do three things: make the AI features actually work on the flagship reseller (aicredits.in) with the models the product is committed to (Gemma and Ling 3.0 VL Flash), implement the UI/UX Transformation Report in full, and re-audit everything that remained. Seven commits landed in this cycle; every one is live-verified, test-covered, and pushed.

15	4	6	5
findings catalogued	P1 critical	P2 major	P3 minor

The headline result of this cycle: **the photo-scan feature - the product’s core promise - was completely broken on aicredits.in and is now fully working.** Root cause was measured, not guessed: the gateway hangs server-side on *response_format: json_schema* requests (zero tokens, connection dropped around 120 seconds), and it kills any non-streaming completion at a 30-second wall that real structured scans (40-115 seconds of generation across the Gemma vision family) can never fit under. Both of the app’s two scan paths sent exactly those request shapes. The fix - ship the schema as instruction text, answer in *json_object* mode, and stream the response, folding the SSE body back into the chat-completions envelope - is verified live through the app’s own code path on three Gemma models and four photos, including the repo’s Indian thali and a pizza.

The second headline: the UI/UX Transformation Report’s Waves 1 and 2 are implemented in full, and Wave 3 is partially landed (assistant, training). The 17-size ad-hoc type scale is now a 7-token scale with a runtime contrast gate in CI; 36 Alert.alert call sites collapsed into one toast system plus ten justified destructive confirms; skeletons, empty states, haptics, a unified large-title header, one Badge/Button/Field/ProgressRing/Sheet primitive layer, a global FAB bottom sheet, and the Hevy-pattern set table are all live. The assistant’s knowledge-question tool misroute - a live-user-visible AI bug found in browser end-to-end testing - is fixed and verified against the live gateway.

What remains is honest and bounded. Four P1 findings: reseller web-lookup is blocked by the gateway’s own tool policy (403, documented and in need of graceful degradation), Ling 3.0 VL Flash’s vision path is broken on the gateway side (text-only works; any image 500s - not app-fixable, needs error copy that stops users retrying a permanently failing model), the saved-foods screen has no in-app writer feeding it, and the label/receipt one-shots still travel non-streaming (fine on fast models today, one slow model away from the 30-second wall). The remaining Wave 3 screens (onboarding consolidation, food write-surface, scan-result sheets, profile sub-pages) and Wave 4 hardening are scheduled in the roadmap with effort estimates.

The five issues that most deserve the next working session, in order

- **P1-1 - Reseller web-lookup is gateway-blocked.** Aicredits' workspace tool policy 403s the web_search tool, so scan-time refinement and the barcode "Other..." lookup can never succeed there. The app should detect the tool-policy 403 and degrade honestly instead of failing per-item.
- **P1-2 - Ling 3.0 VL Flash vision is broken gateway-side.** Every image request 500s at 30 seconds while text-only requests succeed; the app's error copy currently invites a retry that will never succeed.
- **P1-3 - saved_meals has no writer.** The saved-foods screen is fed only by backup import; the DayTimeline save action writes logging_shortcuts instead. The feature is an orphan for normal users.
- **P1-4 - Label/receipt one-shots are non-streaming.** gemma-3-12b completes them in 3.6 s today, but gemma-3-27b and gemma-4-31b generation times exceed the gateway's 30 s non-streaming wall. The scan fix should be extended to the vision one-shot family.
- **P2-1 - Onboarding is still 22 screens.** The consolidation to 12 (Chapter 8.1 of the UI/UX report) is the largest single remaining trust win for new users.

Gates at HEAD: **1,266/1,266 unit tests green** (116 files, up from 733/87 at the first audit's baseline), TypeScript strict typecheck clean, ESLint --max-warnings=0 clean, node-purity check clean (19 RN-free packages), Playwright e2e 24 passed + 2 pre-existing fixme skips, and a 23-screen browser walkthrough with zero console errors, zero page errors, and zero failed asset requests. Every commit in the cycle was pushed immediately after its gates ran green.

2. Scope, Method and Severity Model

The audit ran four tracks, three of them read-only. **Track 1 - live AI verification:** the aicredits.in API was probed directly (model catalogue, wire behaviours, timeout walls) and then exercised through the application's own compiled inference modules - the same buildOpenAIRequest, VISION_WIRE_SCHEMA, and validatePayload code the shipped app runs - against the repo's three test photos plus one generated pizza photo, across every vision-capable model in the allowed set. A second pass ran the full user journey in a real browser: backup restore, provider configuration through the settings UI, photo scans through the camera flow, and streaming assistant conversations.

Track 2 - browser walkthrough: an automated Playwright walk restored the app from the e2e backup fixture and visited all 22 routes across the five tabs and every pushed screen, capturing console errors, page errors, and failed network requests at each step, with screenshots archived. **Track 3 - code audit:** targeted review of the inference, scan, and data layers for swallowed errors, unwired guards, and reseller-path gaps, cross-referenced against the first audit's 101 findings to verify which are fixed, partially fixed, or resurfaced. **Track 4 - gates:** the full test suite, typecheck, lint, node-purity, and e2e runs, plus bundle and corpus measurements.

Severity is unchanged from the first audit: **P0** launch-crashing, data-loss, or key-leak defects (none found); **P1** critical - a flagship feature broken, a trust violation, or a guaranteed failure path; **P2** major - a significant functional or UX problem with a workaround; **P3** polish. Every finding carries file-level or live-probe evidence, user impact, a concrete fix, and an effort estimate (S under a day, M one to three days, L more than

a week). Findings are additionally tagged [NEW], [FIXED this cycle], [PARTIAL], or [RESURFACED] against the first audit.

3. AI and Reseller Reliability Audit

3.1 The measured gateway contract

Direct probing of aicredits.in with the working credential established a precise behavioural contract, each row verified with multiple requests. These are properties of the gateway, not of the app - but the app must live inside them, and at baseline it violated all three.

Gateway behaviour	Measured evidence	Consequence for the app
json_schema response_format hangs	Zero streamed tokens; connection dropped ~120 s; HTTP 500 or terminated stream on every attempt (3 models)	The app's primary scan path (strict structured output) can never complete
~30 s wall on non-streaming completions	Every non-streaming completion exceeding ~30 s of generation returns a bare HTTP 500 "Internal Server Error" at 30.0-30.8 s	Real structured scans (40-115 s of generation) die before the model finishes
Streaming + json_object works	HTTP 200, finish=stop, 1.6-4.8 k output tokens, 34-114 s total; usage included in stream chunks	The one request shape that reliably completes a structured scan
web_search tool blocked by policy	POST /v1/responses with tools:[web_search] returns 403 "Tool ... is blocked by this workspace's server tool policy"	Scan-time web refinement and barcode "Other..." lookup can never succeed on this host (P1-1)
Ling 3.0 VL Flash vision adapter broken	Text-only chat: 200 in 10.9 s. Any image payload: 500 at ~30 s, reproducible across all test photos	One of the two allowed model families cannot scan at all on this host (P1-2)

Table 3.1: The aicredits.in gateway contract, measured 2 October 2026.

3.2 The fix: gateway wire mode

runScanAttempt in apps/mobile/src/inference/pathA/calls/scan.ts now detects custom base URLs and re-shapes every gateway attempt before it is sent: the JSON schema ships as instruction TEXT (buildOpenAIRequest's instructionSchema route, response_format json_object) instead of the strict json_schema mode, and the body carries stream: true so the request escapes the 30-second non-streaming wall. Because React Native's fetch cannot read a body incrementally - and because the measured gateways buffer the whole completion anyway - the SSE body is read to completion and folded into the chat-completions envelope every downstream parser already understands (foldSseToEnvelope, a pure function with its own test coverage). A new GATEWAY_SCAN_TIMEOUT_MS of 180 seconds covers real generation time while the AbortController remains the hang guard. Official provider endpoints keep the strict json_schema route they implement correctly.

A second fix landed in the payload repair layer (packages/pipeline/src/repair.ts): instruction-schema scans let models answer null for three fields the strict wire never sees nullled. `uncertainty_reason` null now lands on the neutral enum member "none"; preparation-block fat enums with out-of-vocabulary values ("none" for `intrinsic_fat`) land on "unknown"; and `fallback_macros_at_estimate` null becomes a zeroed Macros object - ignored on the database-hit path, and a visible flagged zero on the miss path rather than a fabricated number. With those three normalisations, every Gemma scan observed in testing passes the full VisionPayload contract including the repair pass.

3.3 Live verification matrix

Every vision-capable model in the allowed set was run through the application's own `runScanWithFallback` against the repo's photos. The assistant path was verified in the browser with streaming replies on a real conversation. The pizza photo was generated for this cycle (the repo's three test photos are all Indian dishes: a Bengali telebhaja plate, a South-Indian banana-leaf sadya, and the steel thali).

Model	Photo	Result	Latency	Output	Contract
google/gemma-3-12b-it	steel thali	PASS	40.0 s	2,108 tok / 7 items	validatePayload PASS
google/gemma-3-12b-it	banana-leaf sadya	PASS	43.1 s	2,170 tok / 5 items	honest null grams
google/gemma-3-12b-it	Bengali snack plate	PASS	30.7 s	1,620 tok / 4 items	validatePayload PASS
google/gemma-3-12b-it	pizza (generated)	PASS	40.7 s	1,593 tok / 4 items	"Pepperoni pizza", composite_dish
google/gemma-3-27b-it	steel thali	PASS	108.2 s	2,231 tok / 7 items	validatePayload PASS
google/gemma-4-31b-it	steel thali	PASS	114.0 s	4,817 tok / 10 items	spotted the gulab jamun
inclusionai/ling-3.0-flash-vl	steel thali	FAIL	30.8 s	HTTP 500	gateway-side vision failure (P1-2)
google/gemma-3-12b-it	assistant chat	PASS	3.4 s first reply	streamed text	knowledge answer correct post-fix

Table 3.2: Live scan matrix through the app's own inference code at HEAD.

Scan quality deserves note, not just pass/fail. The thali scan returns a scene-level "Indian mixed thali" classification at 0.95 identification confidence with per-bowl decomposition - exactly the behaviour the v1.3.0 honesty prompt was designed to produce. The banana-leaf sadya scan returns honest null gram estimates with low portion confidence (the prompt's "no reliable scale reference" clause) instead of invented precision. The pizza scan decomposes into dough/sauce/cheese/pepperoni with null grams for the whole unscaled pie. In the browser, the thali scan produced nine review rows of which six matched the bundled corpora (including a Dal Makhani and a USDA rice row) and three carried honest AI-estimate badges. In other words: the full perception-to-database pipeline works end to end on the flagship reseller, with the uncertainty visible at every step the product promises it.

3.4 The assistant tool-misroute fix

Browser end-to-end testing surfaced a subtle AI bug: knowledge questions ("How much protein is in 100g of cooked toor dal?") triggered the `get_nutrition_summary` tool - which returns the user's OWN logged totals, a non-answer ("Nutrition Summary (today): 0 kcal/0g") - while the model's actual text answer was discarded when tool JSON was detected in the stream. Both halves are fixed in 6d23927: the assistant system prompt (now versioned `assistant-v1.1`) scopes `get_nutrition_summary` to the user's diary only and instructs direct text answers for knowledge questions, and `parseAssistantReply` now renders the accompanying text and discards the spurious tool call whenever a reply carries both. Live verification: the same question now answers "Cooked toor dal (pigeon pea) typically contains around 9 grams of protein per 100g" in plain streamed text, reproduced twice, with the diary tool correctly NOT called.

4. P1 - Critical (4 findings)

No P0 blockers were found: no launch-crashing defect, no data-loss path in normal flows, no key leakage beyond the provider the user named, and the full test suite is green. Four P1 findings remain, each with a guaranteed or flagship-impacting failure mode.

P1-1 · Reseller AI — Web-lookup is blocked by the gateway's tool policy; refinement and barcode "Other..." fail per item

Evidence	Live probe: POST <code>https://aicredits.in/v1/responses</code> with tools:[{type:"web_search"}] returns HTTP 403 "Tool ... is blocked by this workspace's server tool policy." The app's web-lookup path (<code>apps/mobile/src/inference/pathA/calls/web-lookup.ts:44</code> , <code>packages/prompt/src/web-lookup.ts:97</code>) builds exactly that request; callers at <code>orchestrator.ts:441</code> (scan-time refinement), <code>:682</code> and <code>:918</code> (receipt/barcode paths) each surface it as a failed lookup.
Impact	On the flagship reseller, the D16 scan-time refinement that replaces AI-estimate rows with cited label data never runs, and the barcode "Other..." flow dead-ends after the local corpus miss. The feature silently degrades: users keep amber AI-estimate rows that marketing copy says will be replaced, and barcode users get an error whose retry will never succeed.
Fix	Classify the tool-policy 403 as its own failure kind; on first occurrence set a session-scoped refinement-unavailable flag and stop per-item retries; surface one honest notice on the result screen (badge on AI-estimate rows: "no cited source on this gateway") and route barcode "Other..." straight to manual entry. Effort: M

P1-2 · Reseller AI — Ling 3.0 VL Flash vision is broken gateway-side; the app's retry copy sends users into a loop

Evidence	Live probes: text-only chat completes (HTTP 200, 10.9 s); ANY image payload on <code>inclusionai/ling-3.0-flash-vl</code> returns HTTP 500 at 30.0-30.8 s, reproduced across all three repo photos via both the raw wire harness and the app's <code>runScanWithFallback</code> . The failure is classified error-retryable, so the scan UI offers retry.
Impact	A user who selects one of the two model families the product advertises cannot scan at all. Every retry fails identically after 30 seconds; nothing in the UI suggests the model is the problem, so the natural conclusion is "the app is broken" - the worst possible read for the core feature.

Fix	The gateway defect is not app-fixable; the EXPERIENCE is. On a second identical vision 500 for the same model, stop offering retry and show: “This model cannot process photos through your gateway. Switch to a Gemma vision model (e.g. google/gemma-3-12b-it) or try Ling with a text-only question.” Track per-model failure counts in the scan store. Effort: S
------------	--

P1-3 · Data layer — The saved_meals table has no in-app writer; the saved-foods screen is fed only by backup import

Evidence	Task-5 verification during Wave 1B: no code path inserts into saved_meals; the DayTimeline “Save meal” action writes logging_shortcuts instead (src/components/DayTimeline.tsx). The saved-foods screen reads only the saved_meals table; the e2e verification had to craft a row via the backup fixture to exercise the (now working) Log again button.
Impact	For every user who never imports a backup, the Saved Foods tab is permanently empty no matter how many meals they “save” from the timeline - a feature that looks broken by design. The P1-5 fix made its button work; the screen still has nothing to show.
Fix	Write saved_meals from the meal-detail/timeline save action (or migrate the screen to read logging_shortcuts, which IS written). Given the backup schema already carries saved_meals, adding the writer preserves round-trips. Effort: M

P1-4 · Reseller AI — Label, receipt and exercise-estimate one-shots still travel non-streaming through gateways

Evidence	apps/mobile/src/inference/pathA/calls/vision.ts postVisionJson() is non-streaming with LOOKUP_TIMEOUT_MS/ESTIMATE_TIMEOUT_MS budgets. Live measurement: label scan on gemma-3-12b completes in 3.6 s (safe); the same request class on gemma-3-27b/gemma-4-31b exceeds the gateway’s ~30 s non-streaming wall (scan-family generation measured at 108-114 s for these models) and would 500.
Impact	The camera’s label and receipt modes, and the exercise estimate, work on fast models but fail with a bare retryable 500 on the slower Gemma models the app explicitly allows - the same class of silent gateway wall the scan path just escaped.
Fix	Extract the gateway wire mode (instruction-text schema + stream:true + foldSseToEnvelope) into a shared helper and route postVisionJson through it when a baseUrl is set, with the GATEWAY_SCAN_TIMEOUT_MS budget. foldSseToEnvelope is already exported and pure. Effort: M

5. P2 - Major (6 findings)

The major findings are dominated by the unfinished remainder of the UI/UX Transformation Report’s Wave 3 - scope that is designed and specified but not yet shipped. They are catalogued here with the same rigour as defects so the roadmap stays honest about what users still experience.

P2-1 · UI/UX Wave 3 — Onboarding is still 22 screens; the consolidation to 12 is unstated

Evidence	apps/mobile/app/onboarding/ contains 22 files driven by the 20-step flow definition; five purely motivational interstitials (thanks, generate, notifications, potential, trend) remain in the flow per UI/UX report Chapter 8.1.
-----------------	--

Impact	New users tap through five screens that add no information before reaching value - the single largest abandonment risk the transformation report identified, now the largest unshipped item.
Fix	Execute Chapter 8.1: merge sex+workouts, diet+blocker, body metrics, provider+key, and fuse trend+potential into one projection moment; keep the Chrome progress rail; make the plan reveal the hero. Effort: L

P2-2 · UI/UX Wave 3 — Food tab is still a write-surface duplicate of Home's timeline; Home instrument pass unstarted

Evidence	DayTimeline still mounts on both Home and Food ((tabs)/index.tsx and (tabs)/food.tsx); the shortcut strip is not yet one-tap log cards (UI/UX report Chapter 7.1/8.3); the Home ring gained the ProgressRing primitive but not the count-up, press-for-detail, or per-macro inline rings of Chapter 8.2.
Impact	The duplicated timeline doubles render cost and leaves the Food tab without a distinct job; repeat logging (the most performed action) still costs more taps than the Hevy-class pattern the report specifies.
Fix	Remove the Food tab's second DayTimeline mount; rebuild the shortcut strip as one-tap log cards with gram presets and undo toast; add the Home count-up and detail expansion. Effort: M

P2-3 · UI/UX Wave 3 — Scan/result confidence pass and the Fix-flow bottom sheet are unshipped

Evidence	result.tsx (1,091 lines) still uses its three-stage overlay chain for Fix and a full-screen add-ingredient view; camera mode pills are Badge-migrated but the capture-guide arc, first-legend for confidence chips, and inline uncertainty reasons (Chapter 8.4) are absent.
Impact	The single most trust-critical screen keeps its pre-transformation interaction skeleton: fixing a misread item costs three full-screen stages, and uncertainty is still a tap away instead of visible.
Fix	Collapse Fix into one bottom sheet with note field and before/after rows; add-ingredient becomes a sheet with recents; add the confidence legend and inline reasons. Effort: M

P2-4 · UI/UX Wave 3 — Profile is still a 395-line single scroll; diagnostics page not yet the model-caption home

Evidence	profile.tsx remains eight sections in one scroll (the Advanced disclosure landed only inside provider-settings.tsx). The debug-class captions moved behind disclosures (Wave 1B) but the Chapter 8.8 grouped-list sub-pages (Goals, AI Provider, Units/Health, Data, About, Diagnostics) are unbuilt.
Impact	Settings discovery stays poor for exactly the power users the reseller features serve; the surface that should consolidate diagnostics does not exist yet.
Fix	Build the iOS grouped-list sub-page structure per Chapter 8.8 with ItemRow primitives; diagnostics becomes the single home for model ids, base URL, and webhook status. Effort: M

P2-5 · Wave 4 hardening — Dark palette contrast matrix uncomputed; deep-link handler still absent

Evidence	Wave 1A landed a runtime contrast gate for the text tiers it touched (type-scale.test.ts) and retired the light-mode 11px failure (5.01:1 measured), but the full dark-mode pair matrix and CI gate are Wave 4 scope. The nutai:// scheme is declared in app.config.ts and promised in README, with no handler registered anywhere in code (unchanged from the first audit).
-----------------	--

Impact	Dark-mode users ride on a palette whose own source comment admits it was never verified beyond the pairs now covered; widget/notification taps promised by the deep-link scheme silently do nothing.
Fix	Extend the contrast test to compute every theme pair in both modes as a CI gate; register a router-level nutai:// handler. Effort: M

P2-6 · Hygiene — Deprecated file-system API and a web hit-test quirk on Profile rows

Evidence	Browser console shows “Method deleteAsync imported from ‘expo-file-system’ is deprecated” from the scan cleanup path (src/scan/store.ts / orchestrator.ts). Separately, the Task-3 browser E2E hit a React-Native-Web stacking artifact where the Profile “Provider & key” row (app/(tabs)/profile.tsx:231) was covered by the root container until scrolled into view; it did not reproduce in the final 22-route walkthrough, so it is flagged, not confirmed.
Impact	A deprecation warning in every scan session pollutes the only diagnostics surface developers have; the hit-test quirk, if real, blocks the primary AI settings entry on web.
Fix	Migrate deleteAsync to the expo-file-system/next API (S); reproduce the hit-test case at 390x844 with a fresh profile and fix the stacking order if it reproduces (S). Effort: S

6. P3 - Minor and Polish (5 findings)

P3-1 · Assistant — One unreproduced first-reply double-bubble anomaly

Evidence	During Task-8 browser verification, the first assistant reply rendered twice in a single session; three controlled re-runs (fresh profile, restore, two questions each) were clean, and code inspection found no path that appends a bubble to the list twice.
Impact	If it recurs for users, the flagship AI screen visually duplicates its own answer - a trust oddity even when the content is correct.
Fix	Watch item: add a stable message-id guard when appending streamed completions to the conversation list (dedupe by id). Effort: S

P3-2 · Training — ActiveWorkout polling reduced but not event-driven

Evidence	Wave 1B moved the DB re-read from 1 s to 5 s (ActiveWorkout.tsx); the UI/UX report’s end-state is an event-driven banner with a set-count and a reanimated slide-in.
Impact	A static pill still wakes five times a second-minute against the DB during a workout; battery cost is reduced 5x but not eliminated.
Fix	Subscribe to workout-set writes and update the banner on change; reanimated is deliberately not enabled, so use the existing Animated spring. Effort: S

P3-3 · Tests — Two pre-existing e2e fixme skips remain (BUG-007, BUG-009)

Evidence	Playwright e2e: 24 passed + 2 skipped; the two skips carry fixme annotations from before this cycle and were not investigated.
Impact	Known regression coverage gaps in the slowest-layer suite; the unit suite (1,266 tests) covers the same logic, so risk is bounded.

Fix	Triage the two fixme specs: fix, or delete with a comment if the behaviour is covered elsewhere. Effort: S
------------	---

P3-4 · Fixtures — qa-backup fixture ships an empty goals table

Evidence	e2e/fixtures/qa-backup.json tables.goals is []; Task-5 reported a Home skeleton hang after restore with it (currentGoal() null); the final 22-route walkthrough did NOT reproduce the hang, so the fixture is inconsistent rather than the app broken.
Impact	e2e reliability: any test expecting post-restore Home content depends on timing rather than data; future regressions in the goal-fallback path will flake.
Fix	Regenerate the fixture from a real onboarding run so goals is populated; add a fixture invariant test asserting non-empty goals. Effort: S

P3-5 · Assistant — Reasoning disclosure code-inspected but never live-exercised

Evidence	The reasoning-collapsible (chevron rotate) shipped in Task 8 but google/gemma-3-12b-it emits no reasoning_content through this gateway, so no live stream has exercised it; the unit tests cover the parse path only.
Impact	If a reasoning-emitting model is selected later, an untested UI path lights up for the first time in production.
Fix	One manual verification with a reasoning model (or a synthetic reasoning_content fixture in the e2e mock). Effort: S

7. Fixed This Cycle - Verification Ledger

Every fix below is verified by tests and, where the feature is live, by browser or gateway exercise at HEAD. Commits are on main; the first audit's finding numbers are cited where they map.

Commit	What it fixed	Verification
30d41db	Gateway wire mode: aicredits scans complete (json_schema hang + 30 s wall root-caused; schema-as-text + stream + SSE fold). Repair layer: 3 Gemma drift classes. GATEWAY_SCAN_TIMEOUT_MS=180 s.	Live scans on 3 Gemma models x 4 photos through app's own code; all validatePayload PASS
1c2c4a9	Wave 1A: 7-token type scale, 68 ad-hoc sizes swept, elevation/state layers, textFaint 3.40:1 to 5.01:1 AA, runtime contrast gate + eslint fontSize rule.	17 new tests; source sweep fails CI on off-scale sizes; browser style checks
98ab3f3	Wave 1B: one toast system (36 Alert.alert sites to 0 raw / 34 toasts / 10 destructive confirms), window.confirm killed, KILL list (dead Log again P1-5, x closes, model captions behind disclosures, 1 s polling), P1-8 log-it silent failure.	17 toast tests; browser: Undo physically removes the logged meal
e7aa192	Wave 1C: Skeleton + Empty primitives deployed per the state matrix, haptics map (Table 9.2), unified large-title header with collapse, Home pull-to-refresh.	44 new tests; NN/g placement rules enforced in source sweep

Commit	What it fixed	Verification
eacaa63	Wave 2: Button/Badge/Field/ProgressRing/PressableFX primitives, global FAB bottom sheet on all tabs (radius 20, spring, swipe-dismiss), tab-bar pill morph + selection haptic.	64 new tests; browser FAB sheet verified; zero console errors
6d23927	Assistant: knowledge-question tool misroute (prompt v1.1 + text-over-tool parse), autoscroll + Latest pill, bubble tones, tool-card status badges (Ch. 8.7); first-audit P1-3 stall guard confirmed wired (assistant.tsx:382).	10 new tests; live: toor-dal question answers in text, tool NOT called
ca5b316	Training (Ch. 8.5): Hevy set table with check-to-complete + haptic + auto-advance, 12-button wall collapsed into grouped context menu, plate calculator bottom sheet, log-exercise 4 screens to 2, ItemRow lists.	set-table domain tests; Train tab renders clean in walkthrough
(this cycle, various)	First-audit P1-9 (repo.ts zero direct tests): repo-invariants.test.ts now covers logMeal roundtrip, undo/redo chains, idempotency dedupe. P1-7 chat autoscroll: fixed via Task 8. P1-4 preprocessing mislabel: catch branches now classify by error type.	Suite grew 733 to 1,266 tests, all green

Table 7.1: The cycle's fix ledger - seven commits, each gated and pushed on landing.

8. Performance and Size Audit

8.1 Bundle and corpora

The web export measures 12 MB on disk: 3.8 MB of JavaScript across six entry chunks, a 6.2 MB nutrition corpus (7,928 USDA foods, 14,630 portions, 362 fully-mapped Indian dishes, 542 IFCT rows), the sqlite3 wasm engine, and static assets. The corpus stays lazy behind the boot path - first paint does not wait on the database - and the deep-link fallback server keeps COOP/COEP headers correct so the SharedArrayBuffer path opens. The Wave 2 primitive consolidation removed five chip implementations and four field copies in favour of one primitive per family, which is bundle weight going down while features went up; the dead @expo/ui dependency was confirmed already absent from the dependency tree.

8.2 Boot and runtime

The browser walkthrough booted the app, restored a backup, and visited all 22 routes with zero console errors, zero page errors, and zero failed asset requests - the cleanest runtime read of the app this audit series has recorded. The 56-day strip, timeline, and tab morphs render without dropped-frame warnings; the skeleton-first Home keeps the perceived boot under a second on the dev server.

8.3 Token economy

Live gateway accounting for one scan: 9,402 input tokens (system prompt + schema contract + photo), 1,593-4,817 output tokens depending on model depth, and 30.7-114 s wall time. gemma-3-12b-it is the price-performance leader at ~2.1 k output tokens and 40 s; gemma-4-31b-it trades 2x the latency and 2.3x the tokens for noticeably deeper decomposition (10 items including the dessert bowl). Usage counts ride the SSE

stream and land in the cost ledger with real numbers; unknown model ids still report cost-unknown honestly rather than zero. The assistant's knowledge answers now cost one streaming completion with no tool round-trip - the misroute fix removed a wasted call per knowledge question.

9. Test and Gate Audit

Every gate below ran at HEAD during this audit. The suite grew 73 percent this cycle (733 to 1,266 tests, 87 to 116 files) while staying green at every push - the growth is the transformation's own tests arriving with each wave.

Gate	Result	Notes
vitest run	1,266 / 1,266 green	116 files; +533 tests this cycle (toast, type-scale, skeletons, haptics, badges, rings, sheets, set table, assistant parse, golden prompts)
tsc --noEmit (app + root)	clean	strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes
eslint --max-warnings=0	clean	includes the new fontSize-literal rule from Wave 1A
node-purity check	OK - 19 packages	React-Native-free packages verified after rebuilding all workspace dists
Playwright e2e	24 passed + 2 skipped	skips are pre-existing fixme BUG-007/009 (P3-3)
Browser walkthrough (22 routes)	0 errors / 0 warnings / 0 failed requests	backup-restored state; screenshots archived per screen
Live gateway scans	7 PASS / 1 gateway-broken	Table 3.2; the one FAIL is the Ling vision adapter (P1-2)

Table 9.1: Quality gates at HEAD ca5b316.

10. Fix Roadmap

The roadmap keeps the transformation report's wave structure and interleaves this audit's P1 items at the front. Each wave leaves the app releasable and the suite green.

Wave	Scope	Effort	Done when
A. Reseller honesty	P1-1 tool-policy 403 classification + graceful refinement off-notice; P1-2 per-model vision-failure counters and switch-model copy; P1-4 gateway wire mode for label/receipt/estimate one-shots; P1-3 saved_meals writer	2-3 days	No retry loop can trap a user; label/receipt complete on slow Gemma; saved-foods non-empty after timeline save
B. Onboarding + food flow	P2-1 onboarding 22 to 12 per Chapter 8.1; P2-2 Food write-surface + Home instrument pass (7.1, 8.2, 8.3)	4-6 days	Screen count 52 to ~41; repeat logging is one tap; Food tab has no second timeline
C. Trust surfaces	P2-3 scan/result confidence pass and Fix bottom sheet (8.4); P2-4 profile sub-pages + diagnostics page (8.8)	3-5 days	Fix flow is one sheet; model captions live only in diagnostics; settings scan in half a screen

Wave	Scope	Effort	Done when
D. Hardening	P2-5 dark contrast matrix as CI gate + nutai:// handler; P2-6 file-system migration + hit-test repro; P3 sweep (fixture goals, e2e fixmes, message-id guard, event-driven workout banner, reasoning fixture)	ongoing	Every theme pair AA-verified in CI; deep links resolve; zero deprecation warnings in a scan session

Table 10.1: Four waves from here to the transformation's definition of done.

The definition of done is the transformation report's own: a new screen can be built entirely from primitives and tokens with zero local styling; every interactive element meets 44 pt with a press state and a haptic; every list has loading, empty, and error states; every colour pair passes AA in both themes as verified by CI; and the icon audit finds no text glyph standing in for a symbol anywhere. This cycle closed the hardest prerequisite - the AI features the interface exists to serve now actually work on the gateway the audience actually uses. The remaining distance is scheduled, specified, and measured.